

NAME

wcalc – a natural-expression command-line calculator

SYNOPSIS

wcalc [*options*] [*expression ...*]

DESCRIPTION

wcalc is a command-line calculator designed to accept all valid mathematical expressions. It supports all standard mathematical operations, parenthesis, brackets, trigonometric functions, hyperbolic trig functions, logs, and boolean operators.

wcalc accepts input in a variety of manners. If no mathematical expression is given at the commandline, it will evaluate the contents of an environment variable named *wcalc_input* if one exists. If that variable is not set, wcalc will try to read input from standard input (i.e. piped input). If there is no input from that, wcalc enters "interactive" mode. Interactive mode has more features.

While in interactive mode, detailed information about commands, functions, symbols, and variables can be obtained by executing: **\explain** *thing-to-explain*

OPTIONS**-H or --help**

Prints a help usage message to standard output, then exits.

-E Specifies that numerical output should be in scientific notation.**-EE**

Specifies that numerical output should NOT be in scientific notation.

-PXXX

Sets the "precision", or the number of decimal places displayed, to be XXX. This setting only affects output, not internal representations. If the precision is set to -1, the number of decimal places displayed will depend on the value.

Precision is set to autoadjust (-1) by default.

Example: wcalc -P6

-v or --version

Prints the version number and exits.

-d or -dec or --decimal

Results are printed in decimal (base 10). This option is the default, and does not have a default prefix to indicate that numbers are in base 10.

-h or -hex or --hexadecimal

Results are printed in hexadecimal (base 16). Numbers printed in hexadecimal have a prefix of *0x* unless the **-p** or **--prefixes** option is used.

-o or -oct or --octal

Results are printed in octal (base 8). Numbers printed in octal have a prefix of *0* unless the **-p** or **--prefixes** option is used.

-b or -bin or --binary

Results are printed in binary (base 2). Numbers printed in binary have a prefix of *0b* unless the **-p** or **--prefixes** option is used.

-p or --prefixes

Toggles printing prefixes for hexadecimal, octal, and binary forms.

-l or --lenient

Makes the parser assume that uninitialized variables have a value of zero.

-r or --radians

Toggles whether trigonometric functions assume input (and output) is in radians. By default, trigonometric functions assume input is in degrees.

-q or --quiet

Toggles whether the equals sign will be printed before the results.

-c or --conservative

Toggles accuracy guards. Because of the way floating point numbers are stored in computers, some numbers cannot be represented exactly (such as 0.1). Because of this, calculating with those numbers can produce results that are not *exactly* correct, but are different from the correct answer by a very small value (smaller than the floating point value can represent accurately). For example, the calculation of 1-9-.1 can return an extremely small number that is not zero but is less than what can be represented accurately, and thus for all intents and purposes, it is 0. The accuracy guard feature will round numbers to zero if they are less than the representable accuracy of the floating point number. However, sometimes numbers that small or smaller need to be displayed, and thus the accuracy guard should be turned off. Alternatively, the number of internal bits could be increased, which makes it possible to represent numbers with more accuracy.

-u or --units [type]

Prints units used for conversions; parameter **type** can be: lengths, areas, volumes, masses, speeds, powers, forces, accelerations, temperatures, angles, or pressures. If the parameter is not supplied, all units are printed.

--remember

Toggles whether or not expressions that produce errors are remembered in the history. Does not affect command-line math.

--round= { none | simple | sig_fig }

Wcalc can attempt to warn you when numbers have been rounded in the output display. It has two methods of keeping track---either by using significant figures (*sig_fig*), or by a simple digit-counting algorithm. Rounding in the command-line version is denoted by a tilde before the equals sign (~=). Rounding in the GUI version is denoted by changing the text color to red. In some cases, Wcalc may think that the number has been rounded even if it shouldn't have been necessary (this is because of the way floating point numbers are represented internally).

--dsep=X

Sets the decimal separator character to be X.

--tsep=X

Sets the thousands separator character to be X.

--idsep=X

Sets the input-only decimal separator character to be X.

--itsep=X

Sets the input-only thousands separator character to be X.

--bitsXXXX

Sets the number of bits of memory that will be used to internally represent numbers to be XXXX. The default is 1024. Set higher if you need to work with extremely large or extremely small numbers, set lower if you want to use less memory.

--ints

Toggles whether long integers will be abbreviated or not. This conflicts with engineering notation for large numbers, but not for decimals.

--verbose

Toggles verbose mode, which displays the expression to be calculated before calculating it.

--defaults

Prevents reading the .wcalcrc file.

-C or --color

Toggles the use of color in the commandline output.

USER-DEFINED VARIABLES

Variables are supported and may be assigned using the = operator. To assign a variable use the form:

```
foo = anylegalexpression
```

Thereafter, that variable name is the same as the literal value it represents. Expressions can be stored in variables like this:

```
foo = 'anylegalexpression'
```

Expressions stored this way will be interpreted at evaluation time, rather than assignment-time. Note that these cannot be recursive.

All variables may also be stored with a description of what they are. This description is added in the form of a quoted string after the assignment, like this:

```
foo = 'anylegalexpression' 'description'
```

ACTIVE VARIABLES

Active variables are designed to give a functionality similar to user-defined functions. They are variables that rather than representing a value, represent an expression that is evaluated whenever the variable is evaluated. This expression may contain other variable names. For example, after the following sequence of commands:

```
foo=5
bar='foo+4'
```

The variable *bar* will evaluate to 9, or four more than whatever *foo* evaluates to be. These can be stacked, like so:

```
baz='sin(bar)+foo'
```

In this case, *baz* will evaluate to be 5.15643, or the sin of whatever *foo+4* is plus whatever *foo* is.

To demonstrate the utility of these active variables, here are two functions written by Stephen M. Lawson. The first computes the weekday of a given day (*dy*) in a given month (*mo*) in a given year (*yr*). The value it returns is in the range of 1 to 7, where 1 is Sunday, 2 is Monday, 3 is Tuesday, and so forth.

```
weekday='(((floor((yr - floor(0.6 + 1 / mo)) / 400) - floor((yr - floor(0.6 + 1 / mo)) / 100) + floor((5 * (yr - floor(0.6 + 1 / mo))) / 4) + floor(13 * (mo + 12 * floor(0.6 + 1 / mo) + 1) / 5)) - (7 * floor((floor((yr - floor(0.6 + 1 / mo)) / 400) - floor((yr - floor(0.6 + 1 / mo)) / 100) + floor((5 * (yr - floor(0.6 + 1 / mo))) / 4) + floor(13 * (mo + 12 * floor(0.6 + 1 / mo) + 1) / 5)) / 7)) + 1) + 5 + dy) % 7 + 1'
```

The second function computes what day Easter will be for a given year (*yr*) and returns an offset from March 31st. For example, for the year 2005, it returns -4, which means March 27th. Because of leap-year problems, this only works from the year 1900 to 2099, but is a good demonstration nevertheless.

```
easter='((19 * (yr - 19 * floor(yr / 19)) + 24) - floor((19 * (yr - 19 * floor(yr / 19)) + 24) / 30) * 30) + ((2 * (yr - 4 * floor(yr / 4)) + 4 * (yr - 7 * floor(yr / 7)) + 6 * ((19 * (yr - 19 * floor(yr / 19)) + 24) - floor((19 * (yr - 19 * floor(yr / 19)) + 24) / 30) * 30) + 5) - floor((2 * (yr - 4 * floor(yr / 4)) + 4 * (yr - 7 * floor(yr / 7)) + 6 * ((19 * (yr - 19 * floor(yr / 19)) + 24) - floor((19 * (yr - 19 * floor(yr / 19)) + 24) / 30) * 30) + 5) / 7) * 7) - 9'
```

BUILT-IN SYMBOLS

There are two basic kinds of built-in symbols in wcalc: functions and constants.

FUNCTIONS

The functions supported in wcalc are almost all self-explanatory. Here are the basic descriptions.

sin cos tan cot

The standard trigonometric functions

asin acos atan acot or arcsin arccos arctan arccot or sin⁻¹ cos⁻¹ tan⁻¹ cot⁻¹

The standard arc- trigonometric functions.

sinh cosh tanh coth

The standard hyperbolic trigonometric functions.

asinh acosh atanh acoth or arcsinh arccosh arctanh arccoth or $\sinh^{-1}$ $\cosh^{-1}$ $\tanh^{-1}$ $\coth^{-1}$

The standard arc- hyperbolic trigonometric functions.

log ln logtwo

Log-base-ten, log-base-e and log-base-two, respectively. Remember, you can also construct log-base-X of number Y by computing $\log(Y)/\log(X)$.

round

Returns the integral value nearest to the argument according to the typical rounding rules.

abs Returns the absolute value of the argument.

ceil ceiling floor

Returns the ceiling or floor of the argument.

sqrt cbrt

The square and cube root functions.

rand

Returns a random number between 0 and the number given.

irand

Returns a random integer between 0 and the number given.

fact

Returns the factorial of a number.

Gamma

Returns the value of the Gamma function at that value.

lnGamma

Returns the value of the log Gamma function at that value.

zeta

Returns the value of the Riemann zeta function at that value.

sinc

Returns the sinc function (for sinus cardinalis) of the input, also known as the interpolation function, filtering function or the first spherical Bessel function, is the product of a sine function and a monotonically decreasing function.

CONSTANTS

Wcalc supports a lot of constants. Some are special (like **pi**), and some are simply mathematical or physical constants that have been hardcoded in. The physics constants are taken from <http://physics.nist.gov/constants>, and should all be in predictable SI units.

The value of **pi** is special, as it is calculated to however many bits of precision have been specified with the **\bits** command. The default number of bits is 1024, or a value of:

```
3.14159265358979323846264338327950288419716939937
5105820974944592307816406286208998628034825342117
0679821480865132823066470938446095505822317253594
0812848111745028410270193852110555964462294895493
0381964428810975665933446128475648233786783165271
2019091456485669234603486104543266482133936072602
4914127372458699747248223615028234079551511205588
1168465696731309335738719301105597412739780116660
0823447367841524950037348489795545416453901986117
5727227318713884226435889742120217131949568051423
0839931356624755337162012934002605160185668467703
3122428187855479365508702723110143458240736806341
```

```

7989633389232864603510897727208179195996751333631
1014750579717366267579547177770281431880438556092
9672479177350549251018537674006123614790110383192
5028979233679937836193101666790131879693151725794
3860403036395703382632593537215128964016797694845
3904619615481368332936937026831888367580239969088
9326975278116532822249504103365733859441905164461
4642369403738060905908822203694572794411694624061
6684848934170304346480406820774078369140625

```

Similarly, all values that rely on the value of **pi**, like **mu0**, have the same level of precision. Here is a complete list of the symbols used to represent the constants hardcoded into wcalc:

e The logarithm constant:

2.718281828459045235360287471352662497757247093699959574966

gamma

Euler's Constant: 0.5772156649015328606065120900824024310421

593359399235988057672348848677267776646709369470632917467495

146314472498070824809605040144865428362241739976449235362535

0033374293733773767394279259525824709491600873520394816567

K Catalan Constant: 0.9159655941772190150546035149323841107741

493742816721342664981196217630197762547694793565129261151062

485744226191961995790358988033258590594315947374811584069953

3202877331946051903872747816408786590902

g Acceleration due to gravity: 9.80665 m/s/s

Cc Coulomb's Constant: 8987551787.37

Universal Constants

Z0 or **Zzero**

Impedance of Vacuum: 376.730313461 ohms

epsilon0 or **epsilonzero**

Permittivity of Free Space: 8.854187817e-12 F/m

mu0 or **muzero**

Permeability of Free Space calculated as $4 \cdot \pi \cdot 10^{-7}$.

G Gravitational Constant: 6.67259e-11

h Planck Constant: 6.6260755e-34

c Speed of Light: 299792458

Electromagnetic Constants

muB

Bohr Magnetron: 5.78838174943e-11 J/T

muN

Nuclear Magnetron: 3.15245123824e-14 J/T

G0 Conductance Quantum: 7.748091733e-5 S

ec Elementary Charge: 1.60217653e-19

Kj Josephson Constant: 483597.879e9 Hz/V

Rk Von Klitzing Constant: 25812.807449 omega

Atomic and Nuclear Constants

Malpha

Alpha Particle Mass: 6.6446565e-27 kg

a0 Bohr Radius: 5.291772108e-11 m
Md Deuteron Mass: 3.34358335e-27 kg
Me Electron Mass: 9.1093897e-31 kg
re Electron Radius: 2.817940325e-15 m
eV Electron Volt: 1.602177250e-12 J
Gf Fermi Coupling Constant: 1.16638e-5 GeV⁻²
alpha
 Fine Structure Constant: 7.29735253327e-3
eh Hartree Energy: 4.35974417e-18 J
Mh Helion Mass: 5.00641214e-27 kg
Mmu
 Muon Mass: 1.88353140e-28 kg
Mn Neutron Mass: 1.67492728e-27 kg
Mp Proton Mass: 1.67262171e-27 kg
Rinf
 Rydberg Constant: 10973731.568525 1/m
Mt Tau Mass: 3.16777e-27 kg

Physio-Chemical Constants

u Atomic Mass Constant: 1.66053886e-27 kg
Na or **NA**
 Avogadro's Constant: 6.0221367e23
k Boltzmann Constant: 1.3806505e-23
F Faraday Constant: 96485.3383 C/mol
c1 First Radiation Constant: 3.74177138e-16 W m²
n0 or **nzero**
 Loschmidt Constant: 2.6867773e25 m⁻³
R Molar Gas Constant: 8.314472
Vm or **NAk**
 Molar Volume of Ideal Gas: 22.413996e-3 (m³)/mol
c2 Second Radiation Constant: 1.4387752e-2 m K
sigma
 Stefan-Boltzmann Constant: 5.670400e-8
b Wien Displacement Law Constant: 2.8977686e-3 m K

Random Constants

random
 A Random Value
irandom
 A Random Integer

SPECIAL SYMBOLS

There are some special symbols that wcalc accept as input for compound operations.

@Inf@
 Symbol that represents Infinity

@NaN@

Symbol that represents "Not a Number"

COMMANDS

There are several commands that are supported in wcalc.

\pXXX Sets the "precision", or the number of decimal places displayed, to be XXX. This setting only affects output, not internal representations. If the precision is set to -1, the number of decimal places displayed will depend on the value. The default is -1.

\e or **\eng** or **\engineering**

Rotates between always using scientific notation, never using scientific notation, and choosing to do scientific notation when convenient. Can also take an argument that is one of *always*, *never*, and *automatic* to choose a mode directly.

\help or **?**

Displays a help screen.

\prefs Prints out the current preference settings.

\li or **\list** or **\listvars**

Prints out the currently defined variables.

\r or **\radians**

Toggles between using and not using radians for trigonometric calculations.

\cons or **\conservative**

Toggles accuracy guards. Because of the way floating point numbers are stored in computers, some numbers cannot be represented exactly (such as 0.1). Because of this, calculating with those numbers can produce results that are not exactly correct, but are different from the correct answer by a very small value (smaller than the floating point value can represent accurately). For example, the calculation of $1 - 9 - .1$ can return an extremely small number that is not zero but is less than what can be represented accurately, and thus for all intents and purposes, it is 0. The accuracy guard feature will round numbers to zero if they are less than the representable accuracy of the floating point number. However, sometimes numbers that small or smaller need to be displayed, and thus the accuracy guard should be turned off. Alternatively, the number of internal bits could be increased, which makes it possible to represent numbers with more accuracy.

\p or **\picky** or **\l** or **\lenient**

Toggles variable parsing rules. When wcalc is "picky" it will complain if you use undefined variables. If it is "lenient", wcalc will assume a value of 0 for undefined variables.

\re or **\remember** or **\remember_errors**

Toggles whether or not expressions that produce errors are remembered in the history.

\pre or **\prefix** or **\prefixes**

Toggles the display of prefixes for hexadecimal, octal, and binary output.

\b or **\bin** or **\binary**

Results are printed in binary (base 2). Numbers printed in binary have a prefix of *0b* unless the **\prefixes** command is used.

\d or **\dec** or **\decimal**

Results are printed in decimal (base 10). This option is the default, and does not have a default prefix to indicate that numbers are in base 10.

\h or **\x** or **\hex** or **\hexadecimal**

Results are printed in hexadecimal (base 16). Numbers printed in hexadecimal have a prefix of *0x* unless the **\prefixes** command is used.

\o or **\oct** or **\octal**

Results are printed in octal (base 8). Numbers printed in octal have a prefix of *0* unless the **\prefixes** command is used.

\round *none|simple|sig_fig*

Wcalc can attempt to warn you when numbers have been rounded in the output display. It has two methods of keeping track---either by using significant figures (*sig_fig*), or by a simple digit-counting algorithm. Rounding in the command-line version is denoted by a tilde before the equals sign ($\approx$). Rounding in the GUI version is denoted by changing the text color to red. In some cases, Wcalc may think that the number has been rounded even if it shouldn't have been necessary (this is because of the way floating point numbers are represented internally).

\dsep*X* Sets the decimal separator character to be *X*.

\tsep*X* Sets the thousands-place separator character to be *X*.

\idsep*X*

Sets the input-only decimal separator character to be *X*.

\itsep*X* Sets the input-only thousands-place separator character to be *X*.

\hlimit*X*

Sets the limit (*X*) on the length of the history.

\open *filename.txt*

Loads file *filename.txt*.

\save *filename.txt*

Saves the current session and variable list to a file, *filename.txt*.

\bits*XXXX*

Sets the number of bits of precision that will be used to internally represent numbers to be *XXXX*. The default is 1024. Set higher if you need more precision, set lower if you want to use less memory.

\ints Toggles whether long integers will be abbreviated or not. This conflicts with engineering notation for large numbers, but not for decimals.

\prefs or **\preferences**

Displays the current preference settings.

\convert *unit1 unit2*

Converts the previous answer from *unit1* to *unit2*.

\store *variablename*

Saves the specified variable in the preload file, `~/wcalc_preload`

\explain *object*

Explains the specified object. The object can be a variable, constant, function, or command.

\verbose

Verbose mode displays the expression to be calculated before calculating it.

\del or **\delim** or **\delimiters**

Display delimiters in numerical output.

\cmmod Toggle between C-style modulus operation and a more flexible method.

\color Toggles the use of color in the commandline output.

PREFERENCES

Preferences and settings can be retained between invocations of wcalc by storing them in the file `~/wcalcrc`

The format of the file is that each line is either blank or an assignment. Comments are ignored, and are defined as anything to the right of and including a hash mark (#). Assignments are of the form: *key=value*

The possible keys are:

precision

A number defining the display precision. Equivalent to the **\P** command, where `-1` means "auto" and anything else specifies the number of decimal places. This does not affect the behind-the-

scenes precision.

show_equals

Either true ("yes" or "true") or false (anything else). Equivalent to the **--quiet** argument. Specifies whether answers will begin with an equals sign or not.

engineering

Either "always", "never", or "automatic". Equivalent to the **\engineering** command. Specifies whether answers will be displayed in engineering notation or not.

use_radians

Either true ("yes" or "true") or false (anything else). Equivalent to the **\radians** command. Specifies whether trigonometric functions accept input in radians or degrees.

print_prefixes

Either true ("yes" or "true") or false (anything else). Equivalent to the **\prefixes** command. Specifies whether base prefixes (e.g. 0x for hexadecimal numbers) are used when displaying output.

save_errors

Either true ("yes" or "true") or false (anything else). Equivalent to the **\remember_errors** command. Specifies whether lines that contain a syntax error are added to the history or not.

precision_guard

Either true ("yes" or "true") or false (anything else). Equivalent to the **\conservative** command. Specifies whether the display will attempt to eliminate numbers too small to be accurate (hopefully, these are only errors created by the binary approximation of the inputs).

print_integers

Either true ("yes" or "true") or false (anything else). Equivalent to the **\ints** command. Specifies whether whole integers will be printed un-abbreviated or not. This conflicts with engineering notation for large integers, but not for decimals.

print_delimiters

Either true ("yes" or "true") or false (anything else). Equivalent to the **\delimiters** command. Specifies whether delimiters will be added to output when displaying.

thousands_delimiter

Uses the next character after the equals sign as its value. Equivalent to the **\tsep** command. Specifies what the thousands delimiter is, and can affect output if **print_delimiters** is enabled.

decimal_delimiter

Uses the next character after the equals sign as its value. Equivalent to the **\dsep** command. Specifies what the decimal delimiter is.

input_thousands_delimiter

Uses the next character after the equals sign as its value. Equivalent to the **\itsep** command. Specifies what the input-only thousands delimiter is, and cannot affect output.

input_decimal_delimiter

Uses the next character after the equals sign as its value. Equivalent to the **\idsep** command. Specifies what the input-only decimal delimiter is, and cannot affect output.

history_limit

Either "no", for no limit, or a number. Equivalent to the **\hlimit** command.

output_format

Either *decimal*, *octal*, *binary*, *hex*, or *hexadecimal*.

rounding_indication

Either *no*, *simple*, or *sig_fig*. Equivalent to the **\rounding** command.

c_style_mod

Either true ("yes" or "true") or false (anything else). Equivalent to the **\cmod** command. Specifies whether the modulo operator (%) will behave as it does in the C programming language, or

whether it will use a more flexible method. This only affects modulo operations where negative numbers are involved. As an example, with **c_style_mod** set to true (the default):

```
-340 % 60 == -40; 340 % -60 == 40; -340 % -60 == -40
```

However, with **c_style_mod** set to false:

```
-340 % 60 == -40; 340 % -60 == -20; -340 % -60 == 20
```

color Either true ("yes" or "true") or false (anything else). Equivalent to the **\color** command. Specifies whether the commandline interface will use color in its output or not.

colors[XXX]

This is used to specify the color of specific interface elements in the commandline interface. Valid colors are:

```
(bold)black
(bold)red
(bold)green
(bold)yellow
(bold)blue
(bold)magenta
(bold)cyan
(bold)white
```

The XXX must be one of the following values:

```
conversion_category
conversion_unit
prompt
approx_answer
exact_answer
err_location
err_text
pref_name
pref_val
pref_cmd
status
var_name
var_desc
subvar_name
explanation
```

PRELOAD

Wcalc uses a file, **~/.wcalc_preload**, to store persistent information between instances. Typically, this is used to store variables that are frequently defined. This file can be edited by hand with a standard text editor. There is also a command within wcalc (**\store**) to append a variable definition to the end of this file. Any variable defined in this file is defined and available for use in any subsequent invocation of wcalc.

COPYRIGHT

wcalc is Copyright (C) 2000-2014 Kyle Wheeler.

It is distributed under the GPL, version 2, or (at your option) any later version..

SUGGESTIONS AND BUG REPORTS

Any bugs found should be reported to

Kyle Wheeler at kyle-wcalc@memoryhole.net.